Depth-First Search (DFS)

Lecture : Recursive and Iterative Traversal, Stack Operations, DFS Trees and Applications

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

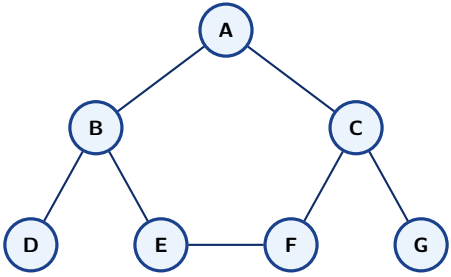
What Is Depth-First Search?

Definition

Depth-First Search is a graph traversal algorithm that explores a path as deeply as possible before backtracking to explore alternative paths.

- Starts from a selected vertex.
- Visits an unvisited neighbor.
- Continues recursively or using a stack.
- Backtracks when no unvisited neighbor remains.
- Uses visited tracking to avoid repetition.

Basic Idea of DFS



Starting at A and processing neighbors alphabetically, one possible DFS order is:

$$\boxed{A, B, D, E, F, C, G}$$

Key Idea

DFS follows one branch deeply before returning to another branch.

Where Is DFS Used?

DFS is useful for:

1. Graph reachability.
2. Connected components.
3. Cycle detection.
4. Topological sorting.
5. Strongly connected components.
6. Maze exploration.
7. Backtracking problems.
8. Finding articulation points and bridges.

DFS vs BFS: Basic Difference

BFS

- Explores level by level.
- Uses a queue.
- FIFO discipline.
- Finds minimum-edge paths.

DFS

- Explores branch by branch.
- Uses recursion or a stack.
- LIFO discipline.
- Supports backtracking.

Why Does DFS Use a Stack?

A stack follows:

LIFO

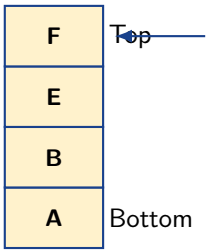
Last In, First Out.

- The most recently explored vertex is handled first.
- DFS remembers the active path.
- When exploration stops, the algorithm returns to an earlier vertex.

Stack Operations

Push: Insert at top.
Pop: Remove from top.

Stack: Visual Representation



Pop Order

$$F \rightarrow E \rightarrow B \rightarrow A$$

Recursive DFS

Recursive DFS uses the program's function-call stack.

1. Visit the current vertex.
2. Select an unvisited neighbor.
3. Call DFS on that neighbor.
4. Continue until no new neighbor exists.
5. Return to the previous function call.

Important

Backtracking occurs automatically when a recursive call returns.

Iterative DFS

Iterative DFS explicitly maintains a stack in the program.

1. Push the starting vertex.
2. Pop a vertex.
3. If not visited, mark and process it.
4. Push eligible neighbors.
5. Repeat until the stack is empty.

Important

Iterative DFS may produce a different order from recursive DFS unless neighbor processing is coordinated.

Why Is a Visited Array Necessary?

Consider a graph cycle:

$$A - B - C - A$$

Without visited tracking, DFS may repeatedly revisit:

$$A \to B \to C \to A \to \cdots$$

Solution

Maintain:

$$visited[v] \in \{0, 1\}$$

Mark each vertex when it is first visited.

Recursive DFS: Main Steps

1. Select a starting vertex.
2. Mark it visited.
3. Process the vertex.
4. Examine adjacent vertices.
5. Recursively visit each unvisited neighbor.
6. Backtrack when all neighbors are processed.

Recursive DFS Pseudocode

Algorithm: DFS(G, u)

```
DFS(u):
  visited[u] = true
  Process(u)
  For each neighbor v of u:
    If visited[v] == false:
      DFS(v)
```

Initial Call

$$DFS(A)$$

Iterative DFS Pseudocode

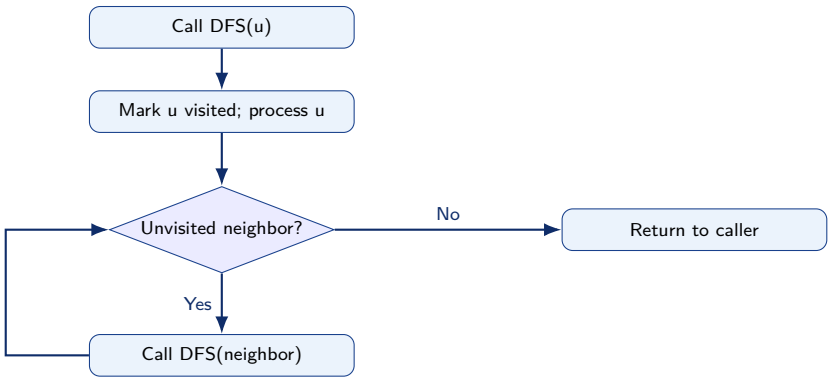
Algorithm: Iterative DFS

```
1.  Create an empty stack
2.  Push(start)
3.  While stack is not empty:
  u = Pop()
  If visited[u] is false:
    visited[u] = true
    Process(u)
    Push eligible neighbors
```

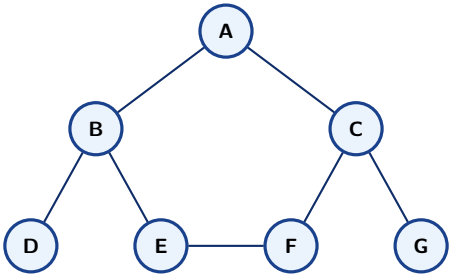
Note

Vertices may be pushed more than once in this simple version. Visited checks prevent duplicate processing.

Recursive DFS Flowchart



Worked Example: DFS from A



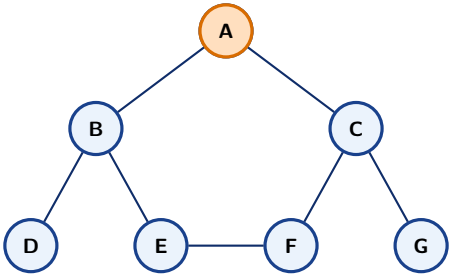
Question

Perform recursive DFS starting from A.
Process neighbors alphabetically.

Reminder

DFS follows the first available unvisited neighbor as deeply as possible.

Step 1: Visit A



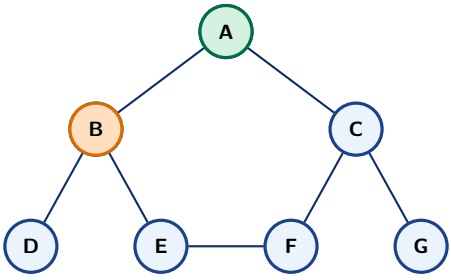
$$Visited = [A]$$

$$Call\ Stack = [A]$$

Next unvisited neighbor:

$$\boxed{B}$$

Step 2: Visit B

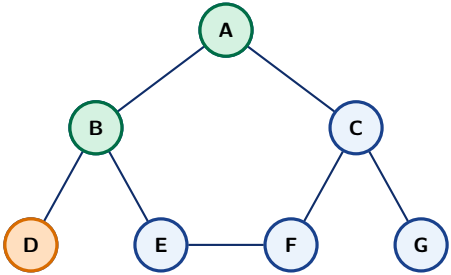


$$Visited = [A, B]$$

$$Call\ Stack = [A, B]$$

B's unvisited neighbors include D and E.
Choose D first.

Step 3: Visit D



$$Visited = [A, B, D]$$

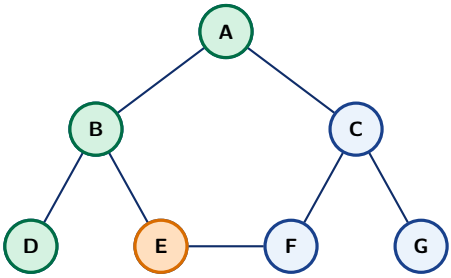
$$Call\ Stack = [A, B, D]$$

D has no unvisited neighbors.

Action

Backtrack from D to B.

Step 4: Visit E

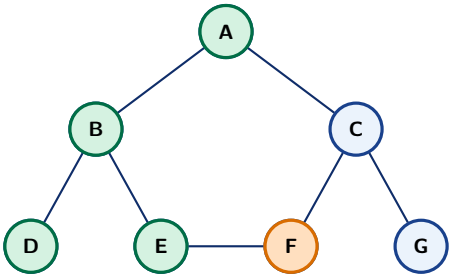


$$Visited = [A, B, D, E]$$

$$Call\ Stack = [A, B, E]$$

E has an unvisited neighbor F.
Continue to F.

Step 5: Visit F

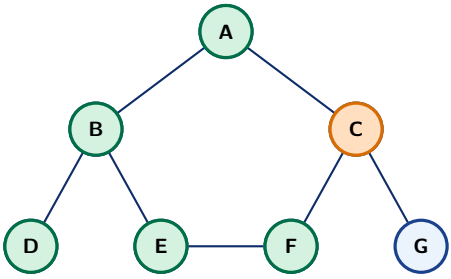


$$Visited = [A, B, D, E, F]$$

$$Call\ Stack = [A, B, E, F]$$

F has an unvisited neighbor C.
Continue to C.

Step 6: Visit C

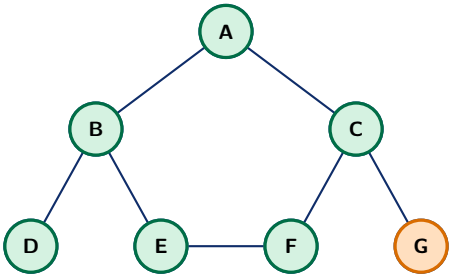


$$Visited = [A, B, D, E, F, C]$$

$$Call\ Stack = [A, B, E, F, C]$$

C has an unvisited neighbor G.
Continue to G.

Step 7: Visit G



$$Visited = [A, B, D, E, F, C, G]$$

G has no unvisited neighbors.
Backtrack until all recursive calls finish.

Complete DFS Dry Run

Step	Visited	Active Call Stack	Action
1	A	A	Start
2	B	A,B	Explore B
3	D	A,B,D	Dead end
4	E	A,B,E	Backtrack, explore E
5	F	A,B,E,F	Explore F
6	C	A,B,E,F,C	Explore C
7	G	A,B,E,F,C,G	Explore G

Final DFS Traversal

$$\boxed{A, B, D, E, F, C, G}$$

Understanding Backtracking

Definition

Backtracking means returning to an earlier vertex when the current vertex has no unvisited neighbors.

For the previous example:

$$A \to B \to D$$

D has no unvisited neighbors.
Return to B and explore E:

$$B \to E \to F$$

Observation

Recursion automatically remembers the earlier vertices.

DFS Order Can Vary

The DFS order depends on the order in which neighbors are examined.
For example:

$$A, B, D, E, F, C, G$$

is valid with alphabetical ordering.
Another valid order may begin:

$$A, C, F, E, B, D, G$$

under a different adjacency order.

Important

Always specify the neighbor-processing order when solving DFS traversal questions.

What Is a DFS Tree?

Definition

A DFS tree contains the edges used to discover each previously unvisited vertex.

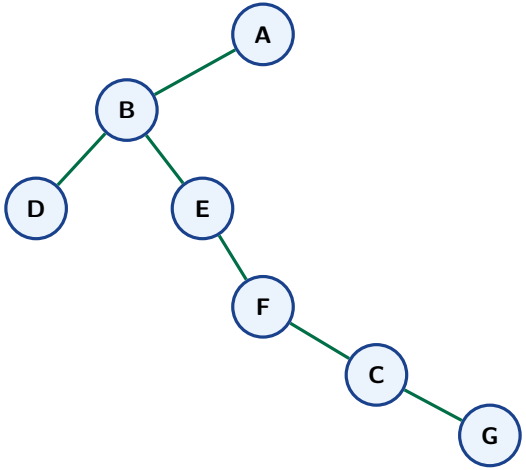
For our example:

$$(A, B), (B, D), (B, E)$$

$$(E, F), (F, C), (C, G)$$

These six edges form a DFS tree containing seven vertices.

DFS Tree: Diagram



$$\boxed{|E_{\text{tree}}| = |V| - 1 = 6}$$

Tree Edges and Non-Tree Edges

During DFS:

Tree Edge

An edge used to discover a previously unvisited vertex.

Non-Tree Edge

An edge that does not cause a new vertex to be discovered.

Example

In the original graph, edge (A, C) is not a DFS tree edge for the chosen traversal order.

DFS Discovery and Finish Times

DFS can maintain two timestamps:

$$d[u] = \text{discovery time}$$

$$f[u] = \text{finish time}$$

- Discovery time is assigned when a vertex is first visited.
- Finish time is assigned after all neighbors have been explored.

Application

These timestamps are useful in topological sorting and edge classification.

Discovery and Finish Time Example

For the DFS tree:

$$A \to B \to E \to F$$

with branch D from B and C–G from F, one valid timestamp assignment is:

Vertex	Discovery	Finish
A	1	14
B	2	13
D	3	4
E	5	12
F	6	11
C	7	10
G	8	9

Recursive DFS Using Adjacency Matrix

We will implement DFS using:

- A two-dimensional adjacency matrix.
- A visited array.
- A recursive function.

Example Graph

$$E = \{(A, B), (A, C), (B, D), (C, E)\}$$

C Program: Recursive DFS

```c
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int visited[MAX];
int vertices;

void addEdge(int u, int v) {
    graph[u][v] = 1;
    graph[v][u] = 1;
}

void dfs(int u) {
    visited[u] = 1;

    printf("%c ", 'A' + u);

    for (int v = 0; v < vertices; v++) {
        if (graph[u][v] == 1 &&
            visited[v] == 0) {
            dfs(v);
        }
    }
}

int main(void) {
    vertices = 5;

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 3);
    addEdge(2, 4);

    printf("DFS Traversal: ");
    dfs(0);
```

Recursive DFS: Output

Expected Output

```
DFS Traversal:  A B D C E
```

Explanation

$$A \to B \to D$$

Backtrack to A.

$$A \to C \to E$$

Recursive DFS: Function Calls

For the previous example:

$$DFS(A)$$

$$DFS(B)$$

$$DFS(D)$$

Return from D and B.

$$DFS(C)$$

$$DFS(E)$$

Observation

The recursion stack stores the active DFS path.

Recursive DFS: Complexity

With an adjacency matrix:

$$\boxed{T(V) = O(V^2)}$$

because every visited vertex scans an entire matrix row.
Recursion depth is at most V:

$$\boxed{\text{Auxiliary Space} = O(V)}$$

The graph matrix itself occupies:

$$\boxed{O(V^2)}$$

Iterative DFS Using a Stack

Iterative DFS avoids recursive calls by using an explicit stack.

Algorithm

1. Push the starting vertex.
2. Pop a vertex.
3. Skip it if already visited.
4. Otherwise mark and process it.
5. Push its unvisited neighbors.

Important

The order of pushing neighbors affects traversal order.

C Program: Iterative DFS, Part 1

```c
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int vertices;

void addEdge(int u, int v) {
    graph[u][v] = 1;
    graph[v][u] = 1;
}

void push(int value) {
    stack[++top] = value;
}

int pop(void) {
    return stack[top--];
}
```

C Program: Iterative DFS, Part 2

```c
void iterativeDFS(int start) {
    top = -1;

    for (int i = 0; i < vertices; i++)
        visited[i] = 0;

    push(start);

    printf("DFS Traversal: ");

    while (top != -1) {
        int u = pop();

        if (visited[u])
            continue;

        visited[u] = 1;
        printf("%c ", 'A' + u);

        for (int v = vertices - 1; v >= 0; v--) {
            if (graph[u][v] && !visited[v]) {
                push(v);
            }
        }
    }

    printf("\n");
}
```

Implementation Note

This simple stack implementation can push duplicate pending vertices. A larger stack or discovery-time marking is required for arbitrary graphs.

C Program: Iterative DFS, Main

```c
int main(void) {
    vertices = 5;

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 3);
    addEdge(2, 4);

    iterativeDFS(0);

    return 0;
}
```

Expected Output

```
DFS Traversal:  A B D C E
```

Iterative DFS: Stack Dry Run

Step	Popped	Stack After Step	Traversal
0	–	A	–
1	A	C,B	A
2	B	C,D	A,B
3	D	C	A,B,D
4	C	E	A,B,D,C
5	E	Empty	A,B,D,C,E

Convention

The rightmost stack element is the top.

DFS with an Adjacency List

For sparse graphs, adjacency lists are generally more memory-efficient.

Representation

Each vertex stores a linked list of its adjacent vertices.

DFS recursively follows the entries in each list.

$$\boxed{T(V, E) = O(V + E)}$$

C Program: List DFS, Part 1

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

typedef struct Node {
    int vertex;
    struct Node *next;
} Node;

Node *adj[MAX];
int visited[MAX];
int vertices;

Node *createNode(int v) {
    Node *p = malloc(sizeof(Node));

    if (p == NULL) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }

    p->vertex = v;
    p->next = NULL;
    return p;
}
```

C Program: List DFS, Part 2

```c
void initialize(int n) {
    vertices = n;

    for (int i = 0; i < n; i++) {
        adj[i] = NULL;
        visited[i] = 0;
    }
}

void addEdge(int u, int v) {
    Node *p = createNode(v);
    p->next = adj[u];
    adj[u] = p;

    Node *q = createNode(u);
    q->next = adj[v];
    adj[v] = q;
}
```

C Program: List DFS, Part 3

```c
void dfs(int u) {
    visited[u] = 1;

    printf("%c ", 'A' + u);

    Node *p = adj[u];

    while (p != NULL) {
        int v = p->vertex;

        if (!visited[v])
            dfs(v);

        p = p->next;
    }
}
```

Important

The traversal order depends on adjacency-list insertion order.

C Program: List DFS, Part 4

```c
void freeGraph(void) {
    for (int i = 0; i < vertices; i++) {
        Node *p = adj[i];

        while (p != NULL) {
            Node *temp = p;
            p = p->next;
            free(temp);
        }

        adj[i] = NULL;
    }
}

int main(void) {
    initialize(5);

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 3);
    addEdge(2, 4);

    printf("DFS Traversal: ");
    dfs(0);
    printf("\n");

    freeGraph();
    return 0;
}
```

Adjacency List DFS: Output

Expected Output

```
DFS Traversal:  A C E B D
```

Explanation

Head insertion places C before B in A's adjacency list.
Therefore DFS explores:

$$A \to C \to E$$

before:

$$A \to B \to D$$

DFS on a Disconnected Graph

Consider:

$$V = \{A, B, C, D, E, F\}$$

$$E = \{(A, B), (B, C), (D, E)\}$$

Starting DFS from A visits only:

$$A, B, C$$

Problem

Vertices D, E and F remain unvisited.

DFS Forest

To visit all vertices:

1. Initialize all vertices as unvisited.
2. Iterate through every vertex.
3. If unvisited, start a new DFS.
4. Repeat until all vertices are visited.

Result

The resulting collection of DFS trees is called a DFS forest.

C Code: Complete DFS Traversal

```c
void dfsAll(void) {
    for (int i = 0; i < vertices; i++)
        visited[i] = 0;

    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            printf("\nNew component: ");
            dfs(i);
        }
    }

    printf("\n");
}
```

Usage

Call `dfsAll()` instead of a single-source `dfs(0)` to traverse all components of an undirected graph.

DFS for Cycle Detection

DFS can detect cycles by examining edges during traversal.

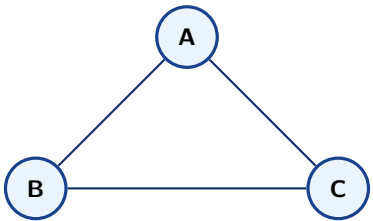
Undirected Graph

If DFS encounters a visited neighbor that is not the current vertex's parent, a cycle exists.

Directed Graph

If DFS encounters an edge to a vertex currently active in the recursion stack, a directed cycle exists.

Undirected Cycle Detection Example



DFS path:

$$A \to B \to C$$

C encounters A, which is visited and is not C's parent.

Cycle detected

C Code: Undirected Cycle Detection

```c
int hasCycle(int u, int parent) {
    visited[u] = 1;

    for (int v = 0; v < vertices; v++) {
        if (!graph[u][v])
            continue;

        if (!visited[v]) {
            if (hasCycle(v, u))
                return 1;
        }
        else if (v != parent) {
            return 1;
        }
    }

    return 0;
}
```

Assumption

This implementation is intended for simple undirected graphs represented by an adjacency matrix.

Directed Cycle Detection

For directed graphs, use three states:

$$0 = \text{Unvisited}$$

$$1 = \text{Active}$$

$$2 = \text{Finished}$$

Rule

An edge from an active DFS vertex to another active vertex indicates a directed cycle.

This method is also useful for checking whether topological sorting is possible.

DFS Time Complexity

Adjacency List

$$\boxed{O(V + E)}$$

Each vertex is visited once, and adjacency entries are examined.

Adjacency Matrix

$$\boxed{O(V^2)}$$

Each visited vertex scans an entire matrix row.

DFS Space Complexity

DFS uses:

- Visited array: $O(V)$.
- Recursion stack: at most $O(V)$.
- Explicit stack: typically $O(V)$ with suitable discovery tracking.

$$\boxed{\text{Auxiliary Space} = O(V)}$$

Graph storage is additional.

Applications of DFS

1. Connected-component discovery.
2. Cycle detection.
3. Topological sorting.
4. Strongly connected components.
5. Maze exploration.
6. Articulation points.
7. Bridges in networks.
8. Backtracking and search problems.

DFS in Maze Solving

A maze can be modeled as a graph.

- Each open cell is a vertex.
- An edge connects neighboring open cells.
- DFS explores a possible route.
- Dead ends trigger backtracking.

Observation

DFS can find a route to the destination, but it does not necessarily find the shortest route.

DFS in Dependency Analysis

Consider tasks:

$$A \to B$$

$$B \to C$$

$$A \to D$$

A DFS-based topological-sort algorithm can produce a valid ordering when the directed graph is acyclic.

Important

Topological sorting applies to directed acyclic graphs (DAGs).

BFS vs DFS: Detailed Comparison

Feature	BFS	DFS
Strategy	Level by level	Depth first
Structure	Queue	Stack / Recursion
Order	FIFO	LIFO
Backtracking	Not central	Fundamental
Unweighted shortest path	Yes	Not guaranteed
List-based time	$O(V + E)$	$O(V + E)$
Matrix-based time	$O(V^2)$	$O(V^2)$
Auxiliary space	$O(V)$	$O(V)$

When Should We Use BFS or DFS?

Problem	Preferred Algorithm
Minimum-edge shortest path	BFS
Level-by-level exploration	BFS
Cycle detection	DFS
Topological sorting	DFS-based method
Backtracking exploration	DFS
Connected components	Either
Reachability testing	Either

Exercise 1: DFS Traversal

Question

Given:

$$E = \{(A, B), (A, C), (B, D), (B, E), (C, F)\}$$

Perform DFS from A with alphabetical neighbor ordering.

Solution

$$\boxed{A, B, D, E, C, F}$$

Exercise 2: DFS Tree

Question

A connected graph contains nine vertices.
How many edges will its DFS tree contain?

Solution

A spanning tree of a connected graph with V vertices contains $V - 1$ edges.

$$9 - 1 = \boxed{8}$$

Exercise 3: Complexity

Question

A graph contains:

$$V = 1000, \qquad E = 1500$$

Which DFS implementation is generally more efficient for neighbor enumeration?

Solution

Adjacency-list DFS.

$$O(V + E)$$

instead of scanning an entire adjacency matrix.

Exercise 4: Cycle Detection

Question

An undirected graph contains:

$$(A, B), (B, C), (C, A)$$

Does it contain a cycle?

Solution

Yes.

$$A \to B \to C \to A$$

Cycle exists

MCQ 1

Which data structure is used by iterative DFS?

1. Queue
2. Stack
3. Heap
4. Hash table

Correct Answer

Option 2: Stack.

MCQ 2

DFS primarily explores:

1. All vertices at the same level first
2. One branch deeply before backtracking
3. Vertices in increasing degree order
4. Vertices by minimum edge weight

Correct Answer

Option 2.

MCQ 3

The time complexity of DFS using adjacency lists is:

1. $O(V^2)$
2. $O(V + E)$
3. $O(E^2)$
4. $O(\log V)$

Correct Answer

Option 2: $O(V + E)$.

MCQ 4

Recursive DFS uses:

1. A queue
2. The function-call stack
3. A priority queue
4. A hash heap

Correct Answer

Option 2.

MCQ 5

Which statement is correct?

1. DFS always finds shortest paths.
2. DFS cannot detect cycles.
3. DFS can generate a spanning tree.
4. DFS requires a priority queue.

Correct Answer

Option 3.

MCQ 6

A DFS forest is useful when:

1. The graph is disconnected
2. The graph has exactly one vertex
3. All edges have equal weight
4. The graph is complete

Correct Answer

Option 1.

DFS: Important Concepts

Concept	DFS Property
Strategy	Depth first
Structure	Stack / Recursion
Discipline	LIFO
Visited tracking	Required
Backtracking	Fundamental
List-based time	$O(V + E)$
Matrix-based time	$O(V^2)$
Auxiliary space	$O(V)$

Lecture Summary

- DFS explores a branch deeply.
- Recursion naturally supports backtracking.
- Iterative DFS uses an explicit stack.
- Visited tracking prevents repeated traversal.
- DFS generates a traversal tree.
- DFS forests cover disconnected graphs.
- DFS supports cycle detection.
- Adjacency-list DFS takes $O(V + E)$ time.
- Adjacency-matrix DFS takes $O(V^2)$ time.

Questions for Revision

1. Define DFS.
2. Explain recursion in DFS.
3. Explain the role of a stack.
4. Perform a DFS dry run.
5. Define backtracking.
6. Construct a DFS tree.
7. Implement recursive DFS in C.
8. Implement iterative DFS in C.
9. Explain DFS forests.
10. Explain DFS-based cycle detection.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 13: Graphs.
2. Thomas H. Cormen et al., *Introduction to Algorithms*, Elementary Graph Algorithms.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*.

Next Lecture

Lecture : Disjoint Sets

- Introduction to disjoint sets.
- Set partitions.
- MAKE-SET operation.
- FIND operation.
- UNION operation.
- Array-based representation.
- Tree-based representation.
- Complete C implementation.
- Applications in graph algorithms.

Student Feedback

Share Your Feedback



Thank you!